Software Design Document (DA-2)

SOFTWARE ENGINEERING • INTERNSHIP & PROJECT COLLABORATION PLATFORM

Student 1: Srishti Jain (24BRS1412)
Student 2: Aakriti Chadha (24BAI1066)
GitHub: github.com/meetsrishti14/Internship-and-Project-Collaboration

Executive Overview: This Software Design Document translates requirements and user stories into a modular, production-ready system design. We formalize software engineering principles (Abstraction, Modularity, Cohesion, Low Coupling), 3-Tier Layered Architecture, Figma UI designs, and key architectural trade-offs for the *Internship and Project Collaboration* platform.

1 Core Design Principles Applied

The system architecture strictly implements the four fundamental pillars of software engineering: **Abstraction, Modularity, High Cohesion, and Low Coupling**.

1.1 Abstraction

How it is applied: Lower-level complexities (such as MongoDB database queries, indexing, third-party authentication algorithms, and SMTP mail protocols) are encapsulated behind clean service interfaces and Data Transfer Objects (DTOs). The presentation tier calls abstract methods like `applyToOpportunity()` without coupling to database drivers or caching layers.

src/services/interfaces/IInternshipService.js • Service Contract Abstraction — JavaScript (ES6)

```
// Abstraction: The Controller and UI consume this abstract contract.
// Underlying database (MongoDB/PostgreSQL) and caching details remain fully encapsulated.
class InternshipService {
  constructor(internshipRepository, notificationService, cacheService) {
    this.internshipRepo = internshipRepository;
    this.notifService = notificationService;
    this.cache = cacheService;
  }

  async applyToOpportunity(studentId, internshipId, applicationData) {
    const isOpen = await this.internshipRepo.isApplicationOpen(internshipId);
    if (!isOpen) throw new Error("This opportunity is no longer accepting applications.");

    const application = await this.internshipRepo.createApplication({
      studentId, internshipId, resumeUrl: applicationData.resumeUrl, appliedAt: new Date()
    });

    await this.notifService.sendReceiptNotification(studentId, internshipId);
    return application;
  }
}
```

1.2 Modularity

How it is applied: The application is decomposed into cohesive, self-contained functional modules: `AuthModule`, `InternshipModule`, `ProjectModule`, `ReviewModule`, and `NotificationModule`. Each module encapsulates its dedicated routes, controllers, validation schemas, and services, enabling parallel development and isolated testing.

src/modules/internships/internship.routes.js • Self-Contained Module Router — Express.js Router

```
const express = require('express');
const router = express.Router();
const { authenticateToken, requireRole } = require('../../middleware/auth.middleware');
const { validateApplication } = require('./internship.validation');
const internshipController = require('./internship.controller');

// Modular Route Definitions encapsulated within the Internship domain boundary
router.get('/', internshipController.getOpportunities);
router.get('/:id', internshipController.getOpportunityById);
router.post('/:id/apply', authenticateToken, requireRole('STUDENT'), validateApplication,
internshipController.submitApplication);
router.post('/create', authenticateToken, requireRole('ORGANIZATION'), internshipController.createOpportunity);

module.exports = router;
```

1.3 High Cohesion

How it is applied: Each component is designed adhering to the *Single Responsibility Principle (SRP)*. Classes and modules perform one well-defined task:

- **Controllers:** Strictly handle HTTP request parsing, status codes, and JSON response formatting. They contain zero business logic or SQL/NoSQL queries.
- **Services:** Solely manage business validation rules, workflow orchestration, and role authorization.
- **Repositories:** Exclusively execute database transactions and data retrieval via Mongoose ODM.

src/modules/projects/project.controller.js • High Cohesion Presentation Controller — JavaScript (Express)

```javascript
// Highly Cohesive Controller: Focuses STRICTLY on HTTP transport & status mapping
const projectService = require('./project.service');

exports.createProjectWorkspace = async (req, res, next) => {
  try {
    const { title, description, techStack, visibility } = req.body;
    const creatorId = req.user.id; // Extracted from verified JWT

    const project = await projectService.initializeWorkspace({
      title, description, techStack, visibility, creatorId
    });

    return res.status(201).json({
      success: true,
      message: "Project workspace initialized successfully",
      data: project
    });
  } catch (error) {
    next(error); // Delegates error formatting to centralized error middleware
  }
};
```

1.4 Low Coupling

How it is applied: Modules interact via standard REST API contracts, Dependency Injection, and an asynchronous Event Bus rather than direct hardcoded dependencies. When milestones are submitted, `ProjectService` emits a domain event without knowing the subscriber details.

src/events/eventBus.js • Loosely Coupled Event-Driven Architecture — Node.js EventEmitter

```javascript
const EventEmitter = require('events');
class AppEventBus extends EventEmitter {}
const eventBus = new AppEventBus();

// In ProjectService.js → Emits event without knowing who handles it (Zero Direct Coupling)
eventBus.emit('project:milestoneSubmitted', {
  projectId: "proj_101",
  milestoneTitle: "Sprint 1 Architecture Signoff",
  submittedBy: "24BRS1412",
  facultyMentorId: "fac_909"
});

// In NotificationService.js → Independent listener; changes do not impact ProjectService
eventBus.on('project:milestoneSubmitted', async (eventPayload) => {
  await emailProvider.sendMentorAlert(eventPayload.facultyMentorId, eventPayload.milestoneTitle);
  await inAppNotificationRepo.createAlert(eventPayload);
});

module.exports = eventBus;
```

> **Architectural Benefit:** This design allows swapping out the email notification provider or adding real-time WebSocket push notifications with zero modifications to project or internship business logic.

Students: Srishti Jain & Aakriti Chadha
Diagram: Draw.io System Architecture

2 High-Level Architecture

The platform follows a **3-Tier Layered Client-Server Architecture** utilizing the **Service-Repository Pattern**. Below is the updated, formal architecture diagram created in Draw.io.

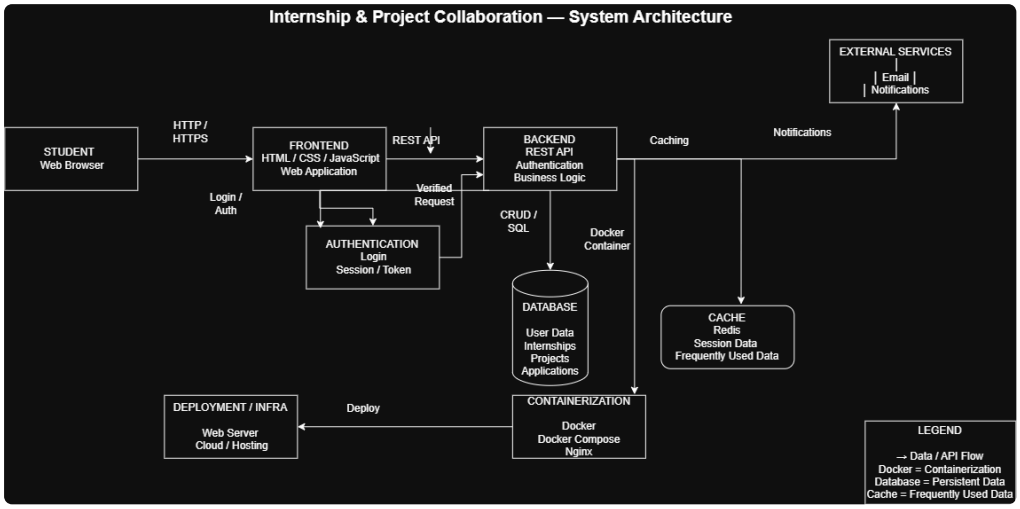


Figure 2.1: High-Level 3-Tier Layered Architecture Diagram (Source: `docs/design/system_architecture.drawio`)

2.1 Chosen Architectural Style: 3-Tier Layered Architecture

Tier / Layer	Key Technologies	Core Function & Responsibilities
1. Presentation Tier	React SPA, HTML5, Vanilla CSS Design System, Axios	Renders user interfaces for Students, Mentors, and Organizations; manages client-side form validation and state.
2. Gateway & Security	Nginx Reverse Proxy, Express Gateway, JWT RBAC	Terminates SSL/TLS, manages CORS headers, enforces rate limiting, and validates cryptographic auth tokens.
3. Application Services	Node.js, Express.js, Modular Domain Services	Executes business logic, access rules, data processing, team invitations, and event orchestration.
4. Persistence & Storage	MongoDB Atlas, Redis Cache, Cloud Document Storage	Stores structured document collections, caches active tokens and search filters, and archives resume PDFs.

2.2 Architectural Justification ("Why?")

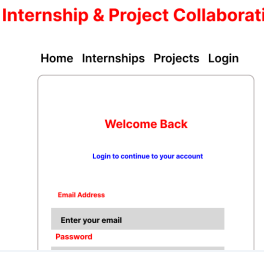
- **Separation of Concerns:** Isolates UI presentation from business logic and database persistence, ensuring UI redesigns require zero backend changes.
- **Parallel Development Velocity:** Frontend and backend engineers work against clean REST contracts concurrently.
- **Horizontal Scalability:** Stateless Node.js containers scale dynamically behind Nginx with zero session lock bottlenecks.
- **Isolated Testability:** Business services can be unit-tested independently with mock database repositories.

Students: Srishti Jain & Aakriti Chadha
Tool: Figma UI Design System

3 User Interface Design & UX Decisions

The UI design in Figma follows modern UX heuristics: visual hierarchy, high contrast ratios, progressive disclosure, and contextual feedback.

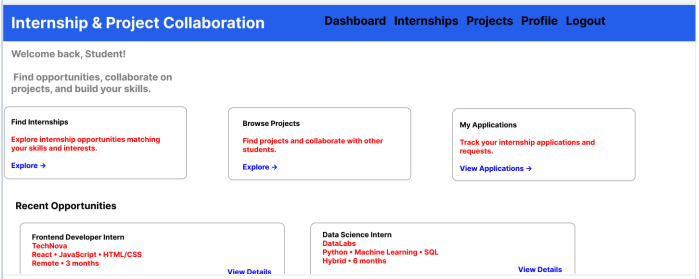
Screen 01 • Secure Login & Role Selector



Key UX Features:

- Distraction-free centered card layout.
- Role-aware credential authentication (Student/Mentor/Org).
- Instant client-side validation with password recovery.

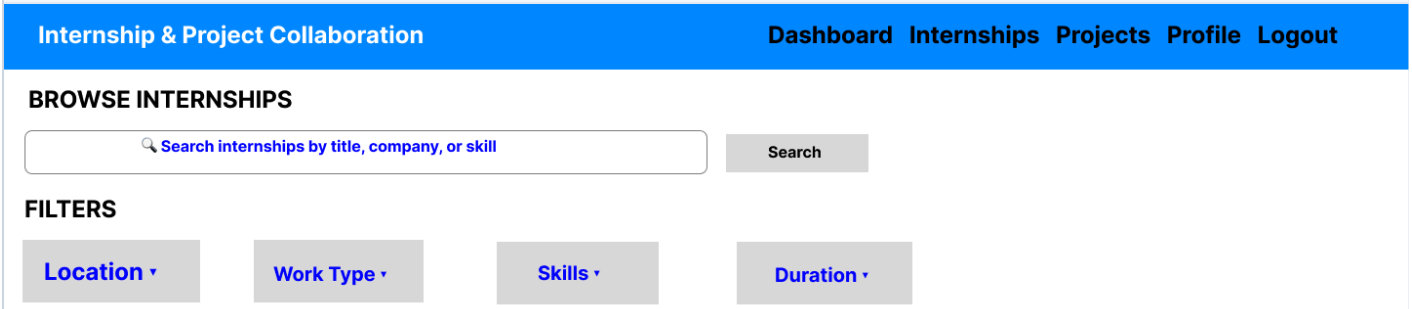
Screen 02 • Student Dashboard Overview



Key UX Features:

- Summary cards (Active Apps, Projects, Matches).
- Personalized greeting with contextual next steps.
- Direct navigation to project workspaces and team milestones.

Screen 03 • Browse Internships & Faceted Filtering



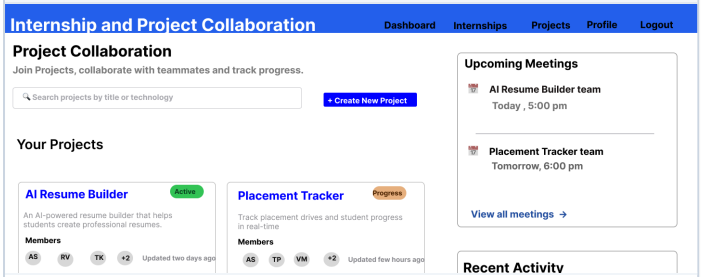
Key UX Features:

- **Faceted Multi-Tag Filters:** Instant client-side filtering by Location (Remote/On-site), Skill Tags, and Duration.
- **Card-Based Opportunity Summaries:** Clean visual hierarchy displaying role title, stipend, deadline, and tech stack badges.
- **One-Click Modal Drawer:** Allows students to review full job specifications and submit applications without page reloading.

Design Tokens & Consistency: Standard color tokens (Primary Blue: `#2563eb`, Success Green: `#16a34a`, Slate Neutral: `#1e293b`) ensure a unified, professional user experience across all modules.

Students: Srishti Jain & Aakriti Chadha
Focus: Collaboration, Profile & Faculty Oversight

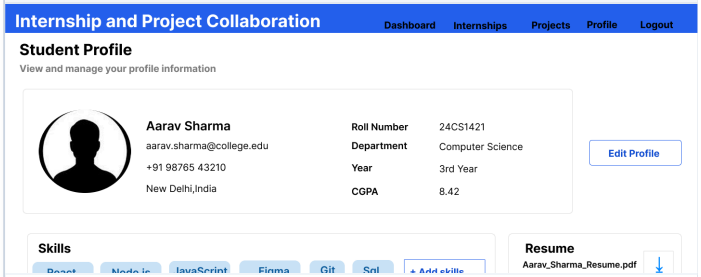
Screen 04 • Project Collaboration Workspace



Key UX Features:

- Project card grid with team avatars and status tags.
- Upcoming meeting widgets and activity feeds.
- Kanban task overview with visual progress bars.

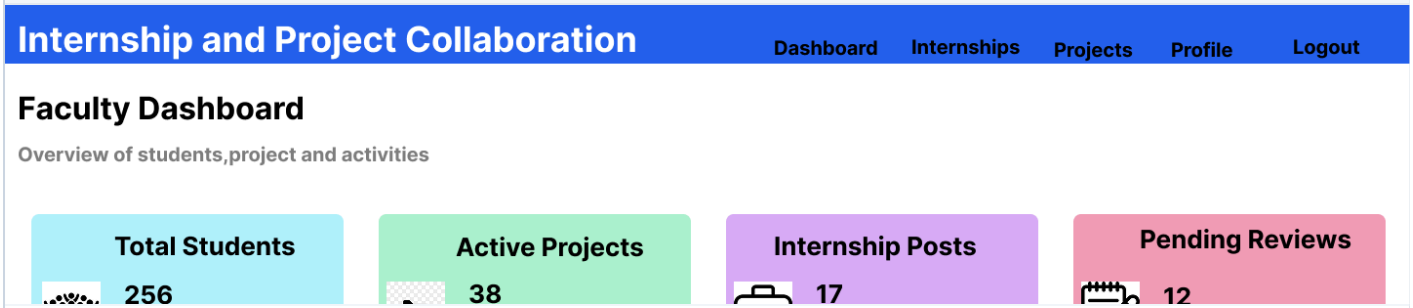
Screen 05 • Student Profile & Portfolio Manager



Key UX Features:

- Academic credentials and dynamic skills matrix.
- Interactive PDF resume uploader and live preview.
- Application tracker with status pills (Review, Selected).

Screen 06 • Faculty / Mentor Review Dashboard



Key UX Features:

- **KPI Metric Tiles:** Total Students (256), Active Projects (38), Internship Posts (17), and Pending Reviews (12).
- **Supervision Table:** Tabular overview of supervised students, assigned projects, and milestone progress.
- **Quick Actions:** Direct one-click shortcuts to post internships, review milestones, and export assessment reports.

3.1 How We Made the UI User-Friendly

- **Reduced Cognitive Load:** Workflows are organized into role-specific views so users see only contextually relevant actions.
- **Visual Affordance & Status Colors:** Badges use standardized cues (Green = Active/Selected, Amber = In Progress/Review, Red = Closed).
- **Accessibility & Contrast:** Strict compliance with WCAG AA standards (high contrast text > 4.5:1 ratio and keyboard navigation).

Students: Srishti Jain & Aakriti Chadha
Section: Architectural Decisions

4 Key Design Decisions & Why

Key architectural decisions were evaluated against viable alternatives to optimize performance, developer agility, and maintainability:

1. RESTful JSON API Architecture over GraphQL
Decision: Adopt standard RESTful JSON HTTP endpoints over GraphQL.
Why: The domain models (internships, projects, applications) have clearly defined boundaries and predictable CRUD operations. REST simplifies edge-level caching via Nginx/CDN, leverages native HTTP status codes (200, 201, 401, 403, 404, 500), and provides straightforward security controls per endpoint without complex query depth analysis.

2. Stateless JWT Authentication with Role-Based Access Control (RBAC)
Decision: Implement JSON Web Tokens (JWT) with embedded role claims (Student, Mentor, Organization) over server-side session cookies.
Why: JWTs are self-contained and cryptographically verified at the API Gateway level. This eliminates database lookup overhead on every authenticated request, enabling horizontal backend scaling across multi-container Docker nodes with zero session synchronization overhead.

3. MongoDB (Document Model) over Rigid Relational Schema
Decision: Utilize MongoDB with Mongoose ODM as the primary data persistence layer.
Why: Internship postings and project workspaces feature flexible, hierarchical data structures (e.g., dynamic skill tags, embedded task checklists, varying application questionnaire responses, and nested feedback comments). MongoDB's polymorphic document format accommodates rapid feature iterations without requiring complex, locking database table migrations.

4. Component-Based Frontend Architecture (React SPA)
Decision: Construct the client as a modular Single Page Application using reusable UI components.
Why: Encapsulating UI elements (e.g., `<OpportunityCard />`, `<KanbanBoard />`, `<SkillTagSelector />`) provides predictable state management, zero unnecessary full-page refreshes, and effortless component reuse across Student, Faculty, and Recruiter views.

5. Docker Containerization & Environment Parity
Decision: Containerize the entire application lifecycle using Docker and Docker Compose.
Why: Eliminates the classic "works on my machine" barrier by bundling Node.js runtime, Nginx web server, and local database services into reproducible container images, ensuring identical behavior across Windows, Linux, and cloud production environments.

Trade-off Summary: These choices prioritize developer velocity, horizontal elasticity, clean API boundaries, and responsive client experiences without over-engineering initial delivery.

5 Maintainability & Future Adaptations

The platform is engineered as an **Evolutionary Architecture**. By isolating services and communicating via domain event buses, future features can be integrated seamlessly:

Future Enhancement	Architectural Integration Strategy	Maintainability Impact
AI Recommendation Engine	Subscribe machine-learning worker service to the `application:created` event bus.	Zero changes to existing core backend; AI worker operates asynchronously.
Cross-Platform Mobile App	Consume the existing RESTful API endpoints via Flutter / React Native.	Reuses 100% of backend services, auth layers, and data validation rules.
University SSO / OAuth2	Add Passport.js SAML/OAuth strategy inside the existing `AuthModule`.	Replaces internal credential validation while preserving downstream user IDs.
Microservices Extraction	Spin off `NotificationModule` and `ReviewModule` into discrete containers.	Existing REST routes and service contracts map directly to microservice endpoints.

6 Submission Deliverables & Repository Structure

In accordance with Digital Assignment 2 guidelines, all design assets, editable source files, wireframes, and documentation updates have been committed to the official GitHub repository.

> **Official Repository URL:**
> **https://github.com/meetsrishti14/Internship-and-Project-Collaboration**

Deliverable Item	File Location in Repository	Format / Details
Editable Architecture Diagram	`docs/design/system_architecture.drawio`	Draw.io XML Editable Source
Exported Architecture Diagram	`docs/design/system_architecture.png`	High-Resolution PNG
UI Wireframes (6 Screens)	`docs/design/screens/*.png`	PNG Screen Captures (Figma UI)
Updated Repository Readme	`README.md` (Section: Software Design)	Markdown with embedded assets
Software Design Document	`DA-2(24BRS1412).pdf`	Final Compiled Report (PDF, 7 Pages)

Project Team: Srishti Jain (24BRS1412) • Aakriti Chadha (24BAI1066) • Software Engineering (Review 2 / DA-2)